

Simpler Can Be Better: Lessons from a Neural Decoding Challenge

 medium.com/@amineziad/research-isnt-linear-a-neural-decoding-case-9e83e780d884

Amine Ziad Ounnoughene

February 1, 2026

Press enter or click to view image in full size



Photo by [Marek Pavlík](#) on [Unsplash](#)

Context

I attempted to take part in the IEEE BioCAS 2025 Grand Challenge under the guidance of Dr. Younes Bouhadjar. The performance of our method on solving the task, was not much above the state of the art, consequently in the end we chose not to submit. Still, the experience itself is worth documenting.

What Was the Challenge About?

The challenge was titled:

IEEE BioCAS 2025 Grand Challenge on Neural Decoding for Motor Control of non-Human Primates

At a high level, it targets a small (but critical) piece of a much bigger problem: intra-cortical Brain–Machine Interfaces (iBMIs).

The long-term goal is to improve the lives of patients affected by spinal cord injuries or neurodegenerative disorders such as (Tetraplegia, Amyotrophic Lateral Sclerosis (ALS) or Other severe motor impairments).

Using BMI technologies, neural activity recorded from the motor cortex is decoded into kinematic variables (velocity, acceleration), which can then control prosthetics, exoskeletons, or external devices.

Why Only 100 Neurons?

There are two main approaches to recording neural signals: **wired BMIs**, which offer high data rates but are bulky and sometimes invasive, and **wireless BMIs**, which are safer and more scalable but heavily constrained and noisy.

As stated in the challenge description

Wireless BMIs are difficult to scale beyond data rates of a few tens of Mbps due to increased bit-error rates, limited battery lifetime, and power dissipation constraints within cortical implants of 80 mW/cm²

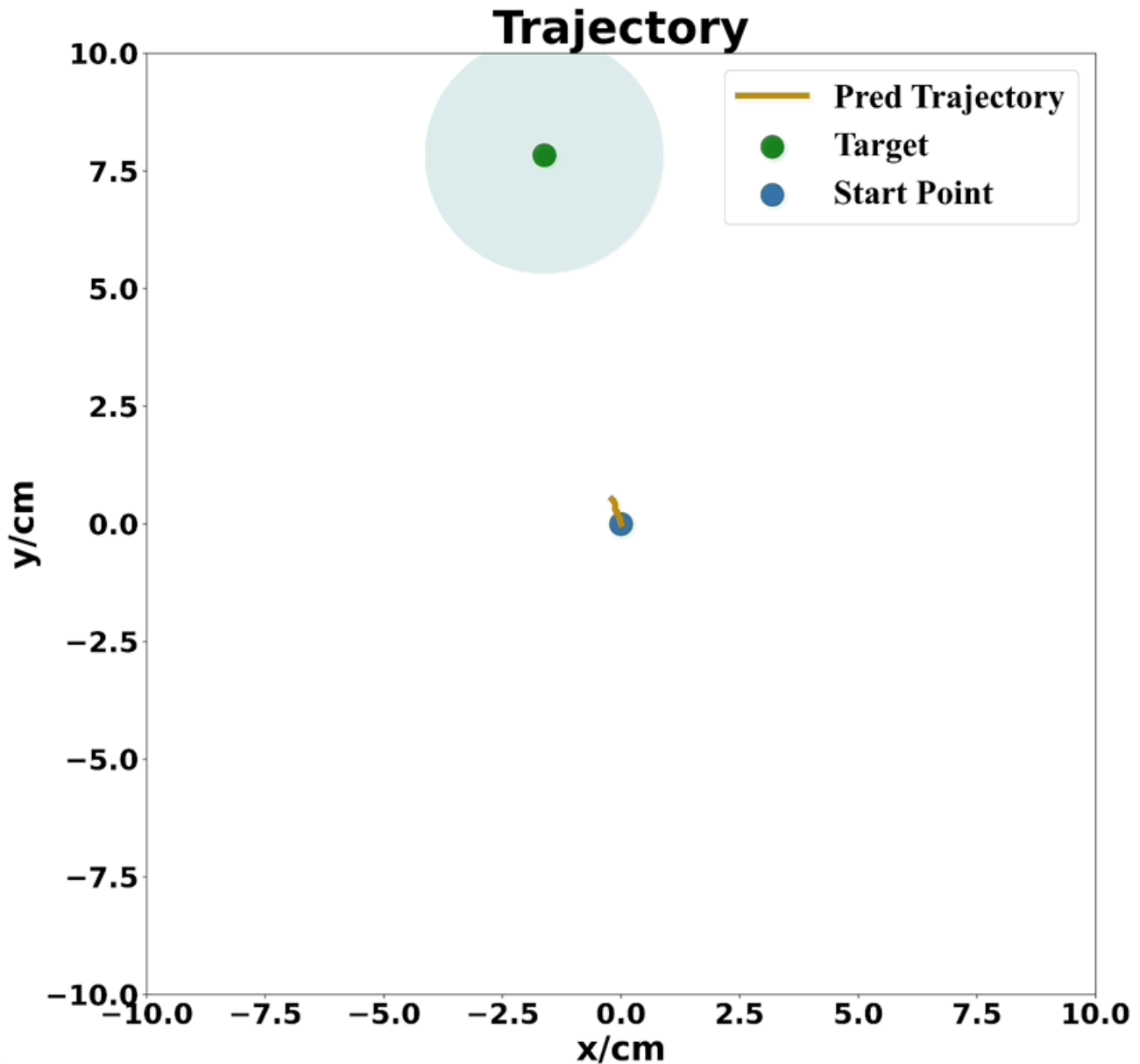
This limits the number of recording channels to roughly 100 neurons.

How to decode meaningful motor behavior? robustly and in closed loop?

Since we do not have access to real-time human brain data to test our decoder model, the challenge has been simplified by using a simulator that acts as a surrogate brain. We train our decoders in this environment to evaluate our methodology

Our **goal** is, from a given starting point, to predict the motion based on the OPS data until the target is reached, under the assumption that the participant always attempts to move in the direction of the target. An example is shown in this figure.

Press enter or click to view image in full size



Trajectory example from [BIOCAS 2025 challenge description](#)

The OPS Closed-Loop Environment

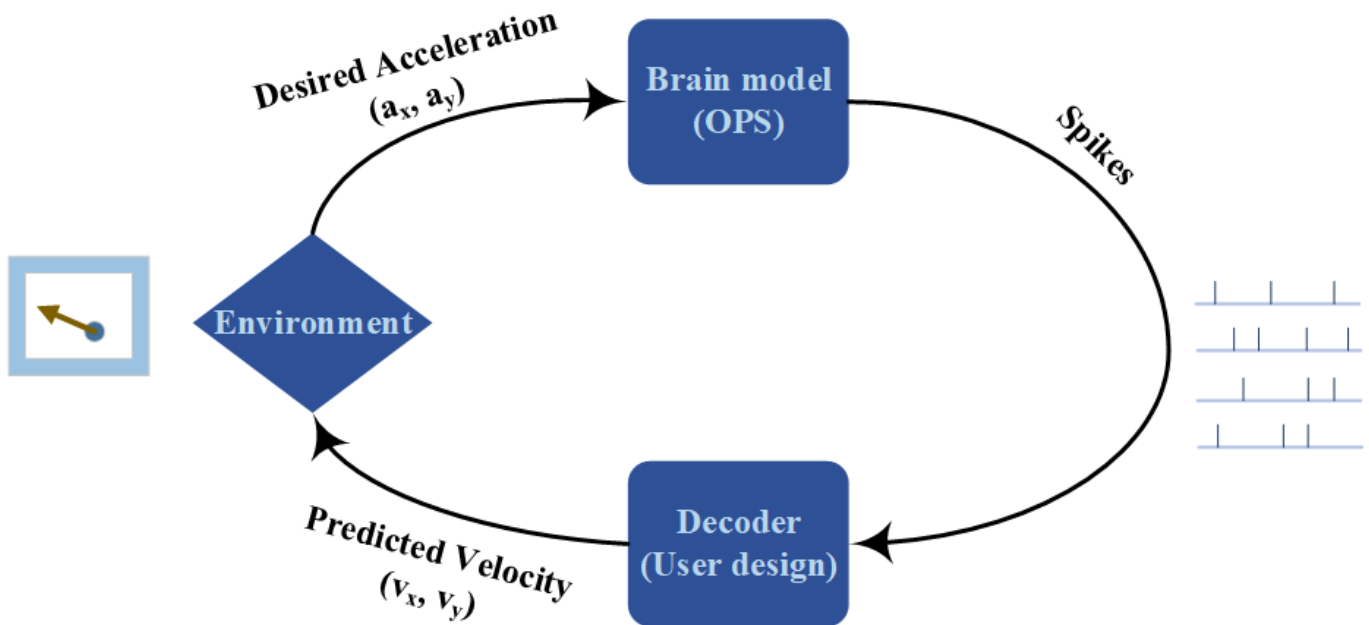
The environment used is the Online Prosthesis Simulator (OPS).

Conceptually, it simulates the brain–body loop:

1. Current position and target are defined
2. A velocity toward the target is computed
3. Velocity induces neural firing probabilities
4. Spikes are generated via Bernoulli sampling
5. A neural decoder estimates velocity from spikes
6. The estimated velocity updates the environment

OPS generates neural activity as if recorded from a micro-electrode array implanted in the motor cortex.

Press enter or click to view image in full size



Closed-loop interaction between OPS and the neural decoder from [BIOCAS 2025 challenge description](#)

How Velocity Is Generated?

The way velocity is generated determines the shape of the resulting trajectory. In this challenge, the velocity is defined using the following approach (shown next), under the assumption that the agent always attempts to move directly toward the target. Let $\mathbf{p}$ denote the position, $\mathbf{t}$ the target, and $\mathbf{v}$ the velocity.

$$\mathbf{d} = \mathbf{t} - \mathbf{p}$$

$$\hat{\mathbf{d}} = \frac{\mathbf{d}}{\|\mathbf{d}\|}$$

$$v_{\text{mag}} = \min\left(k_d \sqrt{\|\mathbf{d}\|}, v_{\text{max}}\right)$$

$$\mathbf{v}_{\text{desired}} = v_{\text{mag}} \cdot \hat{\mathbf{d}}$$

$$\mathbf{v}_{\text{new}} = \mathbf{v} + \alpha (\mathbf{v}_{\text{desired}} - \mathbf{v})$$

Velocity-based description to define the trajectory

What Does “Success” Mean in This Task?

The problem is formulated as an **episodic reaching task**, where each episode corresponds to a single reach. An episode is considered successful if all of the following conditions are met

- Start at least 8 cm from the target
- Reach within a 0.5 cm radius
- Stay there for 50 consecutive steps
- Maximum episode length: 300 steps

The primary objective is to **minimize the total number of steps** required to complete a successful reach

Additional Objectives

In addition to the main objective, the challenge includes two separate evaluation tracks with additional criteria:.

Track 1: Efficiency

This track evaluates computational efficiency, including : Memory footprint (in bytes) & Synaptic operations including Dense operations, MACs (Multiply Accumulate operations for ANNs) and ACs (Accumulate operations for SNNs)

Track 2: Robustness

This track assesses robustness to perturbations, including distribution shifts (electrode drift), where the provided neural data are perturbed, as well as cases in which random probes are set to zero (signal loss).

Methodology

Our First Attempt: Offline Supervised Learning

We trained a spiking neural network (SNN) using backpropagation on recorded spike–velocity pairs generated in the OPS environment (approximately 50 episodes). We got good performance on clean data, but with evere degradation under perturbations (Track 2)

Second Attempt: Continuous Learning

Inspired by:

| A Spiking Neural Network with Continuous Local Learning for Robust Online Brain–Machine Interface by Elijah A. Taeckens and Sahil Shah, 2023

Continuous Local Learning (CLL)

This work introduces a three-factor Hebbian learning rule that enables continuous, online synaptic updates without backpropagation. Learning depends on pre-synaptic activity, post-synaptic activity, and a global modulatory signal. This approach is **online and local**, **memory efficient**, and **robust to neural perturbations**.

We used their implementation as a baseline and evaluated it on this challenge neural files, which include preferred directions of OPS probes.

The model was trained offline on the previously generated dataset using the same continuous learning rule and then tested in closed loop without retraining. It converged easily and remained robust under perturbations, already operating near the task’s theoretical performance limits.

TD-Error Modulated Learning Rate

To explore a possible contribution, we augmented the model with reinforcement-style learning. We treated the decoder as a policy and defined rewards based on cosine similarity between velocity and target direction, as well as the negative weighted distance to the target.

A value-function head was later added and trained using TD error via backpropagation through time. The TD error δ_{TD} was then used to modulate the continuous learning rate, as fellows using a sigmoid function

Learning rate modulation (factor)

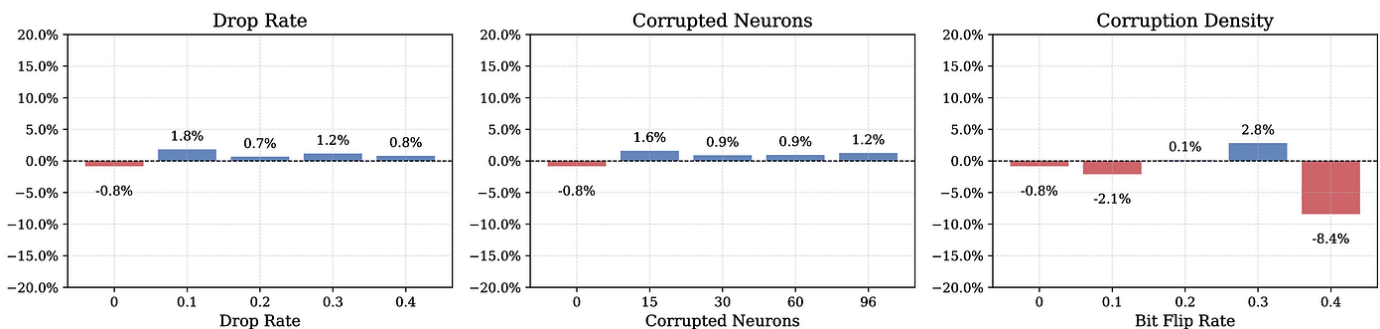
With the following logic : High TD error increased plasticity, while low TD error reduced it.

Why It Didn't Work?

We think it didn't work mainly because we overcomplicated a problem that could have been solved with supervised learning. We added multiple objectives, extra optimization steps, and switched to reinforcement learning, which is known in the literature to be unstable and hard to train. Additionally, The OPS simulator provides a simplified, low-fidelity approximation of the real brain, which probably caused overfitting, and we were also limited by time.

Through this, we learned that sometimes simpler approaches are more reliable, and that the setup and data quality matter a lot. For the future, better simulator fidelity and careful tuning of the learning methods could help get more robust results and stronger scientific insights.

Press enter or click to view image in full size
Gc Neuron1: TD-Modulated Improvement Over Pure CLL



Performance of the TD-modulated learning rate compared to the original continuous local learning method proposed by Taekens *et al.* under neural perturbations (single neural file). Perturbations include dropout, where a small fraction of randomly selected neurons are set to zero; corrupted neurons, where the preferred direction of selected neurons is altered; and random sign flipping of a subset of neurons.

References

- Taeckens, E. A., Shah, S., Journal of Neural Engineering, 2023.
- [IEEE BioCAS 2025 Neural Decoding Grand Challenge](#)